Espectáculos

Pepe Calderón, Fernando Sola, Daniel Ayala, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz

Contenido

1. Requisitos 2
2. Entrevista 2
3. Modelo Conceptual 3
4. Diagrama de clases 3
5. Diagrama de objetos 3
6. Modelo Relacional 4
7. Intensión 4
8. Extensión (fragmento) 5
9. Álgebra relacional 5
10. Modelo Tecnológico (MariaDB) 5
11. Script SQL para crear la base de datos 5
12. Script SQL para la carga inicial de datos 7
13. Consultas 9
14. SQL Avanzado 11

Versión PDF disponible

Espectáculos

1. Requisitos

Catálogo de Requisitos La transcripción que aparece a continuación corresponde a una entrevista realizada al gerente de un teatro para determinar los objetivos y requisitos de una aplicación web que gestione la programación anual de espectáculos y venta de entradas a dichos espectáculos.

2. Entrevista

- Cuestión: Para empezar, hábleme de los espectáculos. Me interesa conocer la información que debe gestionarse por parte de la aplicación web.
  - Respuesta: De acuerdo, los espectáculos que se programan en nuestro teatro pueden ser de distinto tipo (p.e. “Concierto”, “Opera”, ..). Cada espectáculo es de un tipo.
- Cuestión: ¿Qué información desea recoger de los espectáculos?
  - Respuesta: Básicamente, su denominación, tipo, duración y fecha y hora en las que se representan. Un espectáculo puede tener varias representaciones.
- Cuestión: ¿Puede haber más de una representación el mismo día?
  - Respuesta: Sí, hay días con más de una representación pero nunca del mismo espectáculo.
- Cuestión: ¿Algo más acerca de los espectáculos?
  - Respuesta: Se me olvidaba, también quiero guardar el precio. Cada tipo de espectáculo tiene un precio de la entrada.
- Cuestión: ¿El precio varía según tipo de espectáculo?
  - Respuesta: Sí, pero también depende de la zona del teatro donde esté situada la localidad.
- Cuestión: Acláreme esto, por favor.
  - Respuesta: El teatro está dividido en zonas. Las localidades más caras corresponden a la zona mejor situada, es decir la zona de “Patio”. Otras zonas son “1 de Balcón”, “1 de Terraza” etc. Cada zona tiene distinto precio.
- Cuestión: En definitiva ¿el precio depende del tipo de espectáculo y zona del teatro?
  - Respuesta: Así es. También quiero conocer las localidades vendidas para cada representación.
- Cuestión: ¿Hablamos de las entradas?
  - Respuesta: Una entrada es una localidad para una representación. Dentro de cada zona las localidades se identifican por fila y butaca (p.e. “fila 2, butaca 3 de Patio”, “fila 2, butaca 3 de 1 de Balcón”, …)
- Cuestión: ¿Quiere que se puedan adquirir las entradas únicamente a través de la web?
  - Respuesta: No, seguiremos vendiendo entradas en taquilla. Además algunas entadas se regalan como invitación. Necesito saber el medio o canal por el que se han adquirido las entradas ya sea web, taquilla o invitación.
- Cuestión: ¿Qué información quiere almacenar de las entradas?

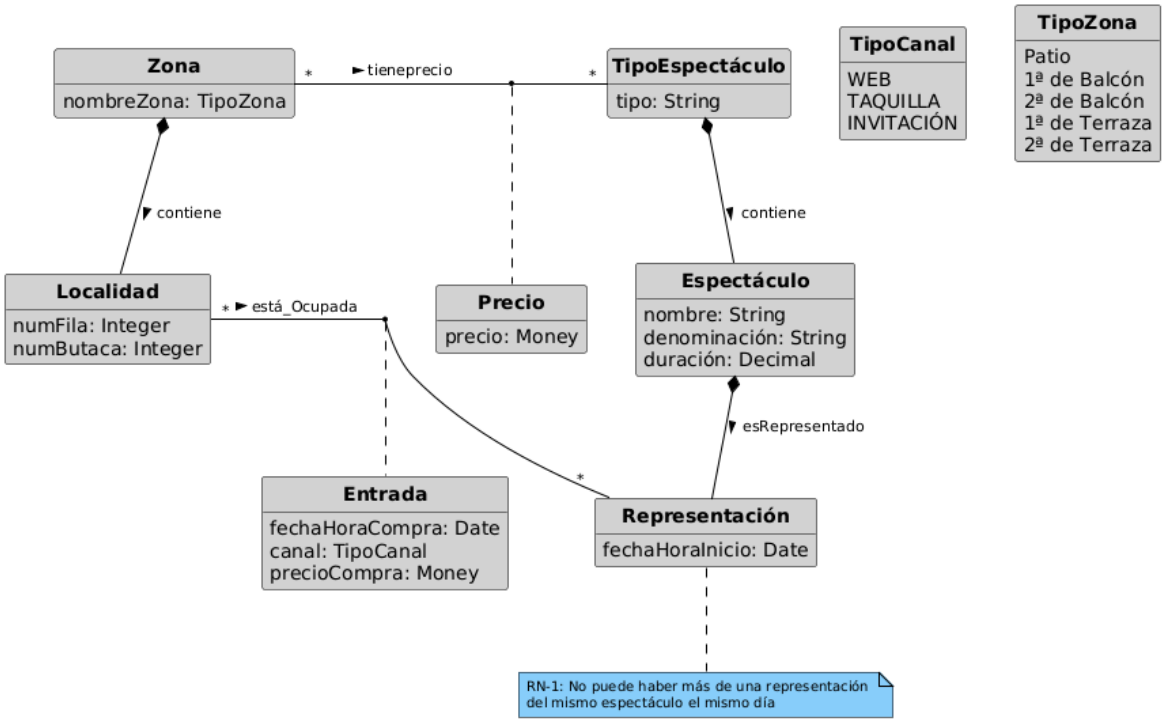
  - Respuesta: La fecha y hora de compra, el canal por el que se ha adquirido, representación, fila y butaca que le corresponde y el precio de compra al que se ha adquirido ya que puede variar con el paso del tiempo.
- Cuestión: ¿Algo más?
  - Respuesta: Creo que en principio es todo. Como ya le comenté, queremos que tanto la planificación de espectáculos como la venta de entradas estén gestionados por la aplicación y se puedan consultar a través de Internet.

3. Modelo Conceptual

Modelo conceptual

4. Diagrama de clases

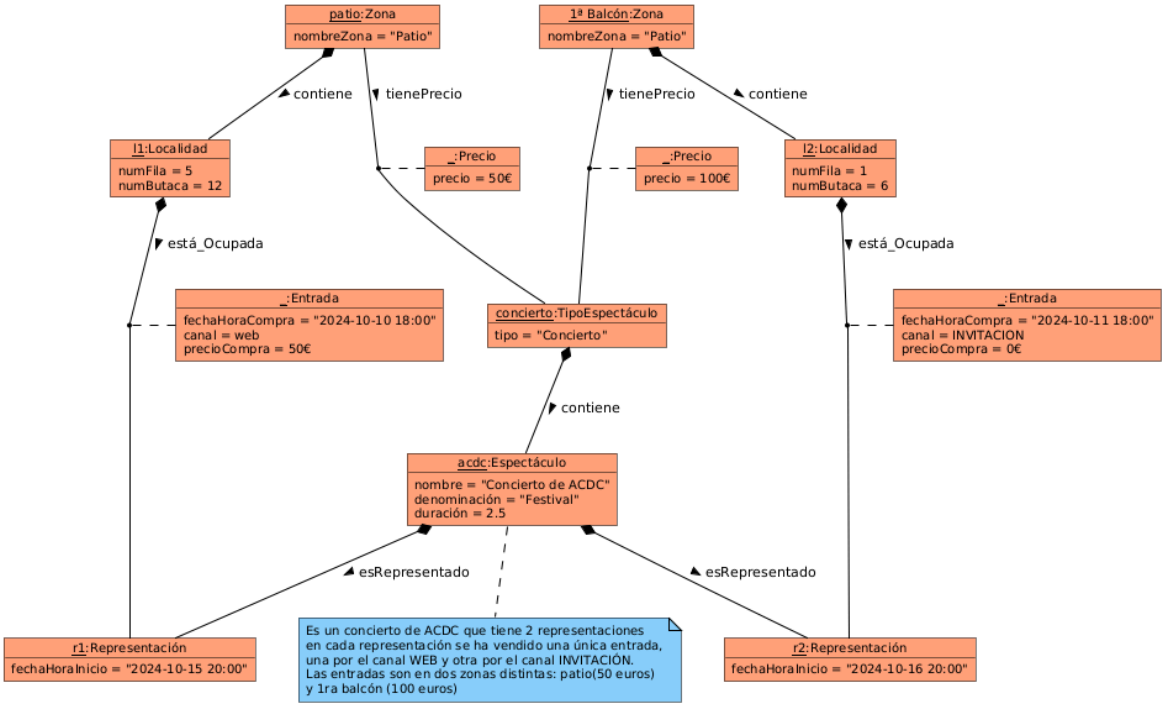
- Estructura por composición: TipoEspectáculo contiene Espectáculo; Espectáculo compone sus Representación; Zona compone sus Localidad.
- Precios: relación M:N entre Zona y TipoEspectáculo materializada por la clase de asociación Precio (importe vigente).
- Entradas y ocupación: relación M:N entre Localidad y Representación materializada por la clase de asociación Entrada (fechaHoraCompra, canal, precioCompra).
- Enumeraciones: TipoCanal {WEB, TAQUILLA, INVITACIÓN} y TipoZona {Patio, 1ª/2ª de Balcón, 1ª/2ª de Terraza} restringen valores.



5. Diagrama de objetos

- acdc:Espectáculo pertenece a concierto:TipoEspectáculo y tiene dos representaciones (r1, r2) en días consecutivos, cumpliendo la RN-1.
- Zonas y localidades: patio y 1ª de balcón contienen las localidades l1 y l2, respectivamente.

- Precios por zona y tipo: cincuenta=50€ para (Patio, Concierto) y cien=100€ para (1ª Balcón, Concierto).
- Entradas vendidas: e1 (WEB) ocupa l1 en r1 por 50€; e2 (INVITACIÓN) ocupa l2 en r2 por 0€; ambas materializan la ocupación (Localidad, Representación).



6. Modelo Relacional

Modelo relacional

7. Intensión

```
TiposEspectaculos(tipoEspectaculoId, tipo)
  PK(tipoEspectaculoId)
Zonas(zonaId, nombreZona)
  PK(zonaId)
Precios(precioId, zonaId, tipoEspectaculoId, precio)
  PK(precioId)
  FK(zonaId) / Zonas
  FK(tipoEspectaculoId) / TiposEspectaculos
Localidades(localidadId, zonaId, numFila, numButaca)
  PK(localidadId)
  FK(zonaId) / Zonas
  AK(zonaId, numFila, numButaca)
Espectaculos(espectaculoId, tipoEspectaculoId, nombre, denominacion, duracion)
  PK(espectaculoId)
  FK(tipoEspectaculoId) / TiposEspectaculos
Representaciones(representacionId, espectaculoId, fechaHoraInicio)
  PK(representacionId)
  FK(espectaculoId) / Espectaculos
  AK(espectaculoId, fechaHoraInicio)
Entradas(entradaId, representacionId, localidadId, fHoraCompra, canal, pCompra)
  PK(entradaId)
  FK(representacionId) / Representaciones
```

```
  FK(localidadId) / Localidades
  AK(representacionId, localidadId)
```

8. Extensión (fragmento)

```
TiposEspectaculos = { (te1, "Concierto") }
Zonas = { (z1, "Patio"), (z2, "Primera Balcón") }
Precios = { (p1, z1, te1, 50), (p2, z2, te1, 100) }
Localidades = { (l1, z1, 5, 12), (l2, z2, 1, 6) }
Espectaculos = { (e1, te1, "Concierto de ACDC", "Festival", 2.5) }
Representaciones = { (r1, e1, "2024-10-15 20:00"), (r2, e1, "2024-10-16 20:00") }
Entradas = { (en1, r1, l1, "2024-10-10 18:00", "Web", 50), (en2, r2, l2,
"2024-10-11 15:00", "Invitación", 0) }
```

9. Álgebra relacional

- Precios por zona y tipo:

$$PZT \leftarrow Precios \bowtie Zonas \bowtie TiposEspectaculos$$

- Localidades por zona/tipo/representación:

$$PZTRL \leftarrow PZT \bowtie Localidades$$

- Entradas por representación:

$$numER \leftarrow \overset{\text{COUNT}(*)}{\gamma} representacionId(Entradas \bowtie Representaciones)$$

- Representaciones con espectáculos:

$$RE \leftarrow Representaciones \bowtie Espectaculos$$

- Número de representaciones por espectáculo:

$$numRE \leftarrow \overset{\text{COUNT}(*)}{\gamma} espectaculoId(Representaciones \bowtie Espectaculos)$$

- Recaudación por representación:

$$Recaudaciones \leftarrow \overset{\text{SUM}(pCompra)}{\gamma} representacionId(Entradas)$$

- Localidades vendidas en r3:

$$LocR3 \leftarrow \overset{\text{COUNT}(localidadId)}{\gamma} representacionId(\underset{representacionId=3}{\sigma}(Entradas))$$

- Entradas por invitación:

$$EntradasInvitacion \leftarrow \underset{canal=\text{ Invitacion}}{\sigma}(RE \bowtie Entradas)$$

10. Modelo Tecnológico (MariaDB)

Modelo tecnológico (MariaDB) Para crear el esquema de la base de datos en MariaDB se puede usar el siguiente script:

11. Script SQL para crear la base de datos

Espectaculos/createDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: script para crear el esquema de la BD
--

DROP DATABASE IF EXISTS EspectaculosDB;
CREATE DATABASE EspectaculosDB;
USE EspectaculosDB;

-- Crear la tabla TiposEspectáculos
CREATE OR REPLACE TABLE TiposEspectaculos (
    tipoEspectaculoId INT AUTO_INCREMENT,
    tipo VARCHAR(255) NOT NULL,
    PRIMARY KEY (tipoEspectaculoId)
);

-- Crear la tabla Zonas
CREATE OR REPLACE TABLE Zonas (
    zonaId INT AUTO_INCREMENT,
    nombreZona VARCHAR(255) NOT NULL,
    PRIMARY KEY (zonaId)
);

-- Crear la tabla Precios
CREATE OR REPLACE TABLE Precios (
    precioId INT AUTO_INCREMENT,
    zonaId INT,
    tipoEspectaculoId INT,
    precio DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (precioId),
    FOREIGN KEY (zonaId) REFERENCES Zonas(zonaId),
    FOREIGN KEY (tipoEspectaculoId) REFERENCES
TiposEspectaculos(tipoEspectaculoId),
    UNIQUE(zonaId, tipoEspectaculoId)
);

-- Crear la tabla Localidades
CREATE OR REPLACE TABLE Localidades (
    localidadId INT AUTO_INCREMENT,
    zonaId INT,
    numFila INT,
    numButaca INT,
    PRIMARY KEY (localidadId),
    FOREIGN KEY (zonaId) REFERENCES Zonas(zonaId),
    UNIQUE (zonaId, numFila, numButaca)
);

-- Crear la tabla Espectáculos
CREATE OR REPLACE TABLE Espectaculos (
    espectaculoId INT AUTO_INCREMENT,
    tipoEspectaculoId INT,
    nombre VARCHAR(255) NOT NULL,
    denominacion VARCHAR(255),
    duracion TIME,
    PRIMARY KEY (espectaculoId),
```

```
    FOREIGN KEY (tipoEspectaculoId) REFERENCES
TiposEspectaculos(tipoEspectaculoId)
);

-- Crear la tabla Representaciones
CREATE OR REPLACE TABLE Representaciones (
    representacionId INT AUTO_INCREMENT,
    espectaculoId INT,
    fechaHoraInicio DATETIME NOT NULL,
    PRIMARY KEY (representacionId),
    FOREIGN KEY (espectaculoId) REFERENCES Espectaculos(espectaculoId),
    UNIQUE (espectaculoId, fechaHoraInicio)
);

-- Crear la tabla Entradas
CREATE OR REPLACE TABLE Entradas (
    entradaId INT AUTO_INCREMENT,
    representacionId INT,
    localidadId INT,
    fHoraCompra DATETIME,
    canal VARCHAR(255),
    pCompra DECIMAL(10, 2),
    PRIMARY KEY (entradaId),
    FOREIGN KEY (representacionId) REFERENCES Representaciones(representacionId),
    FOREIGN KEY (localidadId) REFERENCES Localidades(localidadId),
    UNIQUE (representacionId, localidadId)
);
```

12. Script SQL para la carga inicial de datos

Para cargar los datos de prueba se puede usar el siguiente script:

Espectaculos/populateDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Script para cargar los datos de pruebas
--

-- Datos a partir de la extensión relacional
INSERT INTO TiposEspectaculos (tipoEspectaculoId, tipo) VALUES
  (1, 'Concierto')
;
INSERT INTO Zonas (zonaId, nombreZona) VALUES
  (1, 'Patio'),
  (2, 'Primera Balcón')
;
INSERT INTO Precios (precioId, zonaId, tipoEspectaculoId, precio) VALUES
  (1, 1, 1, 50),
  (2, 2, 1, 100)
;
INSERT INTO Localidades (localidadId, zonaId, numFila, numButaca) VALUES
  (1, 1, 5, 12),
  (2, 2, 1, 6)
;
INSERT INTO Espectaculos (espectaculoId, tipoEspectaculoId, nombre, denominacion,
```

```sql
duracion) VALUES
  (1, 1, 'Concierto de ACDC', 'Festival', '02:30:00')
;
INSERT INTO Representaciones (representacionId, espectaculoId, fechaHoraInicio)
VALUES
  (1, 1, '2024-10-15 20:00:00'),
  (2, 1, '2024-10-16 20:00:00')
;
INSERT INTO Entradas (entradaId, representacionId, localidadId, fHoraCompra,
canal, pCompra) VALUES
  (1, 1, 1, '2024-10-10 18:00:00', 'Web', 50),
  (2, 2, 2, '2024-10-11 15:00:00', 'Invitación', 0)
;

-- Insertar datos generados con IA:
INSERT INTO TiposEspectaculos (tipo) VALUES
  ('Teatro'),
  ('Danza')
;
INSERT INTO Zonas (nombreZona) VALUES
  ('Segunda Balcón'),
  ('Primera Terraza'),
  ('Segunda Terraza')
;
INSERT INTO Precios (zonaId, tipoEspectaculoId, precio) VALUES
  (3, 2, 80),
  (4, 2, 60),
  (5, 2, 40),
  (3, 3, 120),
  (4, 3, 100),
  (5, 3, 80)
;
-- Insertar 3 localidade de cada zona con numero de fila y butaca aleatorios:
INSERT INTO Localidades (zonaId, numFila, numButaca) VALUES
  (3, 5, 12),
  (3, 6, 10),
  (3, 7, 8),
  (4, 1, 6),
  (4, 2, 4),
  (4, 3, 2),
  (5, 1, 6),
  (5, 2, 4),
  (5, 3, 2)
;

-- Insertar 3 espectaculos de cada tipo:
INSERT INTO Espectaculos (tipoEspectaculoId, nombre, denominacion, duracion)
VALUES
  (2, 'El lago de los cisnes', 'Ballet', '02:00:00'),
  (2, 'El cascanueces', 'Ballet', '02:30:00'),
  (2, 'La bella durmiente', 'Ballet', '02:30:00'),
  (3, 'La casa de Bernarda Alba', 'Drama', '02:00:00'),
  (3, 'La vida es sueño', 'Drama', '02:30:00'),
  (3, 'La casa de los espíritus', 'Drama', '02:30:00')
;
-- Insertar 3 representaciones de cada espectaculo:
```

```sql
INSERT INTO Representaciones (espectaculoId, fechaHoraInicio) VALUES
  (2, '2024-10-17 20:00:00'),
  (2, '2024-10-18 20:00:00'),
  (2, '2024-10-19 20:00:00'),
  (3, '2024-10-20 20:00:00'),
  (3, '2024-10-21 20:00:00'),
  (3, '2024-10-22 20:00:00')
;
-- Insertar 3 entradas de cada representacion, si son de canal Invitación el
precio es 0:
INSERT INTO Entradas (representacionId, localidadId, fHoraCompra, canal, pCompra)
VALUES
  (3, 1, '2024-10-10 18:00:00', 'Web', 80),
  (3, 2, '2024-10-11 15:00:00', 'Invitación', 0),
  (3, 3, '2024-10-12 12:00:00', 'Taquilla', 80),
  (4, 2, '2024-10-10 18:00:00', 'Web', 60),
  (4, 3, '2024-10-11 15:00:00', 'Invitación', 0),
  (4, 4, '2024-10-12 12:00:00', 'Taquilla', 60),
  (5, 5, '2024-10-10 18:00:00', 'Web', 40),
  (5, 6, '2024-10-11 15:00:00', 'Invitación', 0),
  (5, 7, '2024-10-12 12:00:00', 'Taquilla', 40),
  (6, 8, '2024-10-10 18:00:00', 'Web', 120),
  (6, 1, '2024-10-11 15:00:00', 'Invitación', 0),
  (6, 2, '2024-10-12 12:00:00', 'Taquilla', 120),
  (7, 3, '2024-10-10 18:00:00', 'Web', 100),
  (7, 4, '2024-10-11 15:00:00', 'Invitación', 0),
  (7, 5, '2024-10-12 12:00:00', 'Taquilla', 100),
  (8, 4, '2024-10-10 18:00:00', 'Web', 80),
  (8, 5, '2024-10-11 15:00:00', 'Invitación', 0),
  (8, 6, '2024-10-12 12:00:00', 'Taquilla', 80)
;
```

13. Consultas

Para crear las consultas SQL de las expresiones en Álgebra relacional se puede usar el siguiente script:

Espectaculos/queries.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Script con las consultas
--

-- Precios por zonas y tipo de espectáculos:
```

```sql
SELECT p.precioId, z.zonaId, t.tipoEspectaculoId, z.nombreZona, t.tipo, p.precio
FROM Precios p JOIN Zonas z
  ON z.zonaId = p.zonaId JOIN TiposEspectaculos t
  ON t.tipoEspectaculoId = p.tipoEspectaculoId
;

-- Localidades disponibles por Zonas:
SELECT z.zonaId, l.localidadId, z.nombreZona, l.numFila, l.numButaca
FROM Localidades l JOIN Zonas z
  ON z.zonaId = l.zonaId
;

-- Precio de las localidades por Zonas y tipo de espectáculo:
-- CREATE OR REPLACE VIEW vPrecioZonaLocalidades as
SELECT p.precioId, z.zonaId, t.tipoEspectaculoId, l.localidadId, p.precio,
z.nombreZona, t.tipo, l.numFila, l.numButaca
FROM Precios p JOIN Zonas z
  ON z.zonaId = p.zonaId JOIN TiposEspectaculos t
  ON t.tipoEspectaculoId = p.tipoEspectaculoId JOIN   Localidades l
  ON z.zonaId = l.zonaId
;

-- Número de entradas vendidas por representación:
SELECT r.representacionId, COUNT(*) numEntradas
FROM Entradas e JOIN Representaciones r
  ON r.representacionId = e.representacionId
GROUP BY r.representacionId
;

-- Representaciones que tienen los espectáculos:
SELECT r.representacionId, e.espectaculoId, e.nombre, r.fechaHoraInicio,
e.duracion
FROM Representaciones r JOIN Espectaculos e
  ON e.espectaculoId = r.espectaculoId
;

-- Número de representaciones que tienen los espectáculos:
SELECT e.espectaculoId, COUNT(*)
FROM Representaciones r JOIN Espectaculos e
  ON e.espectaculoId = r.espectaculoId
GROUP BY e.espectaculoId
;

-- Total recaudado en cada representación:
SELECT e.representacionId, SUM(e.pCompra)
FROM Entradas e
GROUP BY e.representacionId
;

-- Número de localidades vendidas para la representación 'r3':
SELECT COUNT(*)
FROM Entradas e
WHERE e.representacionId = 3
;

-- Entradas por invitación de todas las representaciones agrupadas por
```

```
espectáculo:
SELECT es.espectaculoId, COUNT(*)
FROM Entradas en JOIN Representaciones r
  ON en.representacionId = r.representacionId JOIN Espectaculos es
  ON es.espectaculoId = r.espectaculoId
WHERE en.canal = 'Invitacion'
GROUP BY es.espectaculoId
;
```

14. SQL Avanzado

Para comprobar que la fecha de compra de una entrada es anterior a la fecha de la representación tenemos que usar triggers. El siguiente script muestra cómo se implementa un procedimiento almacenado que hace la comprobación y después se crean dos triggers, uno que hace la comprobación al insertar y otro al actualizar:

Espectaculos/tFechaCompra.sql

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: Compruebar que la fecha de compra en anterior a la fecha de
-- su representación
--

-- Crear el procedimiento almacenado para verificar la fecha de compra
DELIMITER //
CREATE OR REPLACE PROCEDURE pFechaCompraRepresentacion(representacionId INT,
fHoraCompra DATETIME)
BEGIN
   DECLARE fechaHoraInicio DATETIME;
   -- Obtener la fecha de la representación
   SELECT r.fechaHoraInicio INTO fechaHoraInicio
   FROM Representaciones r
   WHERE r.representacionId = representacionId;
   -- Comprobar que la fecha de compra es menor que la fecha de la representación
   IF fHoraCompra >= fechaHoraInicio THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'La fecha de compra debe ser menor que la fecha de la
representación';
   END IF;
END //
DELIMITER ;

-- Crear el trigger para comprobar la fecha de compra antes de insertar en la
tabla Entradas
DELIMITER //
CREATE OR REPLACE TRIGGER tCheckFechaCompraBeforeInsert
BEFORE INSERT ON Entradas
FOR EACH ROW
BEGIN
   CALL pFechaCompraRepresentacion(NEW.representacionId, NEW.fHoraCompra);
END //
DELIMITER ;
```

```
-- Crear el trigger para comprobar la fecha de compra antes de actualizar en la
tabla Entradas
DELIMITER //
CREATE OR REPLACE TRIGGER tCheckFechaCompraBeforeUpdate
BEFORE UPDATE ON Entradas
FOR EACH ROW
BEGIN
   CALL pFechaCompraRepresentacion(NEW.representacionId, NEW.fHoraCompra);
END //
DELIMITER ;
```